API Reference

The complete PhysioGuidance REST API — 46 backend modules grouped by domain, authentication and authorization model, error contract, and where in the code every endpoint lives so you can navigate from doc to source.

PREPARED FOR

PhysioGuidance Engineering Team

DATE

2026-08-17

BASE URL

`https://api.physioguidance.org/api`

PHYSIOGUIDANCE

Riyadh · Saudi Arabia
me-central2 · GCP Cloud Run

Table of contents

1. Base URL and global conventions
2. Authentication and authorization
3. Rate limiting
4. Error contract
5. Locale and i18n
6. Public endpoints (no JWT)
7. Auth and identity
8. User account and preferences (Me)
9. Role requests and trainer applications
10. Courses
11. Sessions
12. Enrollments and attendance
13. Materials
14. Assessments
15. Certificates
16. Course and session assistants
17. Instructor dashboard
18. Student dashboard
19. Reports
20. Payments
21. Bank transfer
22. Invoices
23. Refunds
24. Discounts and campaigns
25. Reconciliation and escalations
26. Finance dashboard and settings
27. Student finance area
28. Admin finance actions
29. Support (tickets, KB, CSAT, SLA)
30. Notifications and badges
31. Points and rewards
32. PDPL admin
33. NELC compliance, audit, and xAPI
34. Zoom integration
35. Storage
36. Admin settings and templates

37. Admin stats and monitoring
38. Backups and website recovery
39. Service status and scheduled maintenance
40. Error monitoring
41. Audit logs
42. Health endpoints
43. Webhook endpoints
44. Internal endpoints
45. Component status matrix
46. Where to look next

---

1. Base URL and global conventions

- Base URL: `https://api.physioguidance.org/api`
- Global prefix `/api` set in `backends/src/main.ts` via `app.setGlobalPrefix(apiPrefix)` (env `API_PREFIX`, default `api`)
- Protocol: HTTPS only
- Content type: `application/json` for request and response bodies
- Multipart bodies: `multipart/form-data` on file uploads (materials, storage, bank-transfer receipts, avatar, captions, assignment submissions)
- Date format: ISO 8601 (`2026-08-17T09:44:34.715Z`)
- Money: integers in halalas (SAR x 100) for gateway integrations; `numeric` strings for DB storage
- Deployed commit: `d396b040`

---

2. Authentication and authorization

2.1 Auth model

- Access token: 15-minute JWT signed with `JWT_SECRET`, stored in an HTTP-only cookie (`sameSite=lax`, `secure` in production)
- Refresh token: 30-day JWT signed with `JWT_REFRESH_SECRET`, hashed server-side in `refresh_tokens` table; frontend cookie holds the token for `/auth/refresh` calls
- Trusted-device chain for reduced 2FA prompts
- Scoped JWTs for trainer-application OTP flow (short-lived, one-purpose)

2.2 Guards

Registered globally in `backends/src/app.module.ts`:

- `ThrottlerGuard` runs first (see section 3)
- `JwtAuthGuard` validates the access token on every non-`@Public()` route
- `RolesGuard` enforces `@Roles(...)` decorators

Per-route decorators:

- `@Public()` bypasses JWT auth
- `@Roles('ADMIN', 'SUPER_ADMIN', ...)` requires one of the listed roles
- `@Throttle({ default: { limit: N, ttl: T } })` tightens the rate limit for a specific route
- `@SkipThrottle()` opts a route out of the global bucket (used on admin dashboards)

2.3 Roles in use

Role	Purpose
`SUPER_ADMIN`	Full access. Promotes and demotes roles including creating other SUPER_ADMINs. Only role that can flip NELC LRS to production.
`ADMIN`	Full admin access except creating SUPER_ADMIN.
`MANAGER`	Course lifecycle, review moderation, instructor roster, reschedule and cancel flows.
`INSTRUCTOR`	Own courses, sessions, students, materials, assessments.
`TEACHING_ASSISTANT`	Assigned sessions, attendance, TA join links.
`STUDENT`	Own enrollments, certificates, profile, payments, refunds.
`FINANCE`	Payments, refunds, invoices, bank-transfer review, reconciliation, discounts.
`SUPPORT`	Support tickets, KB, macros, tags, agent management.

---

3. Rate limiting

Single global bucket declared in `app.module.ts`:

```
ThrottlerModule.forRoot([{ name: 'default', ttl: 60_000, limit: 600 }])
```

That is 600 requests per minute per IP. Sensitive endpoints tighten this per-route with `@Throttle(...)`. Exceeding a limit returns HTTP 429 with a `Retry-After` header.

Endpoint	Effective limit	Rationale
`POST /api/auth/register`	5 / min	Signup spam protection
`POST /api/auth/login`	10 / min	Brute-force protection
`POST /api/auth/reset-password`	5 / min	Reset spam protection
`POST /api/auth/password-setup/complete`	5 / min	Setup-token abuse protection
`POST /api/auth/forgot-password`	3 / min	Email enumeration protection
`POST /api/auth/users/:id/resend-invite`	5 / min	Invite email spam protection
`GET /api/certificates/verify/:id`	30 / min	Public verification abuse protection
`POST /api/payments/initiate`	10 / min	Checkout hammer protection
`POST /api/payments/quote`	10 / min	Cart quote hammer protection
`POST /api/payments/tabby/pre-score`	10 / min	Tabby pre-score fan-out protection
`POST /api/payments/verify/:id`	20 / min	Manual verify polling ceiling
`POST /api/payments/verify-charge/:chargeId`	20 / min	Charge lookup polling ceiling
`GET /api/payments/gateways/health`	20 / min	Gateway health probe protection
`POST /api/payments/abandoned-sweep/run`	5 / min	Manual sweep protection
`POST /api/payments/bank-transfer/submit-draft`	throttled	One-shot receipt submit protection
`POST /api/payments/bank-transfer/proforma/quote`	throttled	Proforma draft abuse protection
`POST /api/payments/bank-transfer/:paymentId/receipts`	throttled	Receipt upload abuse protection
`POST /api/payments/reconciliation/scan`	5 / min	Reconciliation scan cost protection
`PATCH /api/payments/reconciliation/findings/bulk-*`	throttled	Bulk-action abuse protection
`POST /api/payments/reconciliation/escalations/:id/enroll`	throttled	Manual enrollment fan-out protection
`POST /api/payments/reconciliation/escalations/:id/refund`	throttled	Manual refund fan-out protection
`POST /api/payments/reconciliation/cron/stale-reminders`	5 / min	Cron replay ceiling

Endpoint	Effective limit	Rationale
`POST /api/payments/reconciliation/cron/abandoned-sweep`	5 / min	Cron replay ceiling
`POST /api/payments/reconciliation/cron/overdue-invoices`	5 / min	Cron replay ceiling
`POST /api/webhooks/:gateway`	throttled	Signature-verified but rate-capped as defence in depth
`POST /api/tickets/csat/:surveyId`	5 / min	Public survey submit abuse protection
`GET /api/tickets/csat/:surveyId`	10 / min	Public survey view abuse protection
`POST /api/tickets/kb/searches`	60 / min	KB search analytics protection
`POST /api/me/sidebar-badges/:tabKey/viewed`	60 / min	Badge polling ceiling
`POST /api/internal/tasks/dispatch`	throttled	Cloud Tasks dispatch protection

The `/me/export` PDPL export is limited to 1 / 24h in the service layer (not via `@Throttle`).

4. Error contract

Uniform JSON error shape from NestJS's built-in exception filter:

```
{
  "statusCode": 400,
  "message": "human-readable description",
  "error": "Bad Request",
  "timestamp": "2026-08-17T09:44:34.715Z",
  "path": "/api/some/endpoint"
}
```

Status	Meaning	Typical cause
400	Bad Request	DTO validation failed, business rule violated
401	Unauthorized	Missing or expired access token
403	Forbidden	Wrong role, or malware scanner flagged this download
404	Not Found	Resource does not exist, or user does not own it
409	Conflict	Optimistic-lock miss on receipt approval, duplicate resource
425	Too Early	Malware scan not yet complete for this download
429	Too Many Requests	Rate limit exceeded
500	Internal Server Error	Unhandled exception (captured in Sentry)
503	Service Unavailable	Maintenance mode ON, or dependency (DB, Redis) down

---

5. Locale and i18n

- Backend responses are locale-independent (return raw data)
- Frontend renders in EN or AR based on URL path (`/en/...` or `/ar/...`)
- Some endpoints accept a locale query for template selection (invoice generation, email dispatch, WhatsApp template lookup)
- User's default locale stored in `users.preferred_locale`

---

6. Public endpoints (no JWT)

Endpoints decorated `@Public()` — reachable without an access token. Rate limits still apply.

Method	Path	Purpose
GET	`/api/health`	Liveness ping
GET	`/api/health/live`	Cloud Run liveness probe
GET	`/api/health/ready`	Readiness probe (DB + Redis)
GET	`/api/public-stats`	Homepage counter stats
GET	`/api/public-stats/reviews`	Homepage carousel reviews
GET	`/api/public/locale-settings`	Public locale settings
GET	`/api/public/general-settings`	Public general settings
GET	`/api/public/password-policy`	Password policy shown at signup
GET	`/api/public/maintenance-status`	Maintenance-mode banner state
GET	`/api/public/notification-channels`	Enabled notification channels
GET	`/api/public/whatsapp-template/:eventKey/:locale`	Public WhatsApp template body
GET	`/api/public/tax/display`	Public tax display mode (inclusive / exclusive)
GET	`/api/public/service-status`	Live status page snapshot
POST	`/api/public/service-status/view`	Fire-and-forget status-page view tracker
GET	`/api/public/service-status/summary`	Status page transparency footer
GET	`/api/public/service-status/history`	Paginated resolved-incident archive
GET	`/api/public/service-status/banner-incidents`	Cross-dashboard incident banner feed
GET	`/api/public/service-status/active-for-link`	Active incidents picker feed
GET	`/api/courses/public`	Public catalog (active courses)
GET	`/api/courses/public/:id`	Public course detail
GET	`/api/courses/instructors/:id/public`	Public instructor profile
GET	`/api/courses/featured-home`	Homepage featured courses
GET	`/api/courses/:id/ratings`	Public course ratings
GET	`/api/courses/:id/instructor-ratings`	Public instructor ratings for a course
GET	`/api/courses/:id/instructor-aggregated-ratings`	Aggregated instructor reviews across courses
GET	`/api/courses/instructor-reviews/featured`	Featured instructor reviews carousel
GET	`/api/certificates/verify/:id`	Public certificate verification (30/min)
GET	`/api/invoices/verify/:invoiceNumber`	Public invoice verification

Method	Path	Purpose
POST	`/api/auth/register`	New account signup (5/min)
POST	`/api/auth/login`	Sign-in (10/min)
POST	`/api/auth/verify-2fa`	2FA code verification
POST	`/api/auth/resend-2fa`	Resend 2FA code
POST	`/api/auth/refresh`	Refresh access token from refresh cookie
POST	`/api/auth/forgot-password`	Send reset email (3/min)
POST	`/api/auth/reset-password`	Reset password via emailed token (5/min)
POST	`/api/auth/password-setup/validate`	Validate an invite token
POST	`/api/auth/password-setup/complete`	Complete first-time password setup (5/min)
POST	`/api/me/account/cancel-deletion`	Cancel a pending account deletion via emailed token
POST	`/api/trainer-applications/request-otp`	Send OTP to prospective trainer email
POST	`/api/trainer-applications/verify-otp`	Verify OTP, receive scoped JWT
POST	`/api/trainer-applications/submit`	Submit application (scoped JWT required)
POST	`/api/trainer-applications/portal/request-otp`	Portal login OTP for the applicant
POST	`/api/trainer-applications/portal/verify-otp`	Portal OTP verify + returns application snapshot
GET	`/api/trainer-applications/portal/status`	Refresh application status (portal JWT)
POST	`/api/trainer-applications/portal/resubmit`	Resubmit MORE_INFO application from portal
GET	`/api/payments/callback`	BNPL / Moyasar redirect callback landing
GET	`/api/payments/gateways/visible`	Which gateways checkout should show
POST	`/api/payments/dev/mock-state`	Dev-only mock-state setter
GET	`/api/payments/dev/mock-state`	Dev-only mock-state reader
GET	`/api/payments/my-payments`	Buyer-scoped payment list (checkout success page)
POST	`/api/discounts/auto-apply`	Best auto-apply discount for a cart
GET	`/api/discounts/active-banner-campaign`	Active site-wide campaign banner
GET	`/api/discounts/campaigns/report`	Public campaign performance surface

Method	Path	Purpose
GET	`/api/tickets/settings/business-hours`	Public form uses this for online / offline hint
GET	`/api/tickets/settings/ticket-categories`	Public category dropdown
GET	`/api/tickets/kb/categories`	Help Center categories
GET	`/api/tickets/kb/articles`	Help Center articles list
GET	`/api/tickets/kb/articles/:id`	Help Center article detail
POST	`/api/tickets/kb/searches`	KB search analytics tap (60/min)
GET	`/api/tickets/csat/:surveyId`	Retrieve CSAT survey (10/min)
POST	`/api/tickets/csat/:surveyId`	Submit CSAT score (5/min)
POST	`/api/storage/upload`	Trainer document upload for the public application form
GET	`/api/storage/files/:bucket/*`	Local dev file serve (STORAGE_DRIVER=local)
POST	`/api/zoom/webhook`	Zoom webhook (HMAC signed)
POST	`/api/zoom/mock-webhook`	Dev-only Zoom event simulation
POST	`/api/webhooks/:gateway`	Payment gateway webhook dispatcher (HMAC signed)
POST	`/api/webhooks/sentry/alert`	Sentry alert receiver
POST	`/api/payments/reconciliation/cron/daily-scan`	Cloud Scheduler daily scan
POST	`/api/payments/reconciliation/cron/stale-reminders`	Cloud Scheduler stale reminders (5/min)
POST	`/api/payments/reconciliation/cron/abandoned-sweep`	Cloud Scheduler abandoned sweep (5/min)
POST	`/api/payments/reconciliation/cron/overdue-invoices`	Cloud Scheduler overdue invoices (5/min)
POST	`/api/payments/reconciliation/cron/bnpl-status-poll`	Cloud Scheduler BNPL status poll
POST	`/api/internal/tasks/dispatch`	Cloud Tasks dispatcher

7. Auth and identity

Controller: `backends/src/modules/auth/auth.controller.ts` — mounted at `/api/auth`.

Method	Path	Roles	Description
POST	`/auth/register`	Public	Create student account (5/min)
POST	`/auth/login`	Public	Sign in and issue access + refresh cookies (10/min)
POST	`/auth/verify-2fa`	Public	Verify 2FA code and complete login
POST	`/auth/resend-2fa`	Public	Re-issue 2FA code
GET	`/auth/trusted-devices`	Authenticated	List caller's trusted devices
DELETE	`/auth/trusted-devices/:id`	Authenticated	Revoke one trusted device
DELETE	`/auth/trusted-devices`	Authenticated	Revoke every trusted device
POST	`/auth/2fa/toggle`	Authenticated	Enable or disable 2FA on caller's account
GET	`/auth/2fa/status`	Authenticated	Read caller's 2FA state
POST	`/auth/forgot-password`	Public	Send reset link (3/min)
POST	`/auth/reset-password`	Public	Reset password via emailed token (5/min)
POST	`/auth/password-setup/validate`	Public	Validate an admin-issued invite token
POST	`/auth/password-setup/complete`	Public	Complete first-time password setup (5/min)
POST	`/auth/logout`	Authenticated	Sign out and clear cookies
POST	`/auth/refresh`	Public	Refresh access token from refresh cookie
GET	`/auth/me`	Authenticated	Return the authenticated user profile
POST	`/auth/switch-role`	Authenticated	Switch active role among the user's assigned roles
GET	`/auth/my-roles`	Authenticated	List roles held by the caller
POST	`/auth/logout-all`	Authenticated	Revoke every refresh token for the caller
PATCH	`/auth/profile`	Authenticated	Update caller's own profile
PATCH	`/auth/change-password`	Authenticated	Change password with current-password check
GET	`/auth/instructors`	MANAGER, ADMIN, SUPER_ADMIN	Instructor picker

Method	Path	Roles	Description
GET	`/auth/students-search`	MANAGER, ADMIN, SUPER_ADMIN	Student picker for manual enrollment
GET	`/auth/users`	ADMIN, SUPER_ADMIN	Paginated user directory
GET	`/auth/users/export`	ADMIN, SUPER_ADMIN	CSV export of users
GET	`/auth/users/:id/roles`	ADMIN, SUPER_ADMIN	Roles held by a user
POST	`/auth/users/:id/roles`	ADMIN, SUPER_ADMIN	Assign a role to a user
DELETE	`/auth/users/:id/roles/:role`	ADMIN, SUPER_ADMIN	Remove a role from a user
POST	`/auth/users`	ADMIN, SUPER_ADMIN	Create a user with a scoped invite email
POST	`/auth/users/:id/resend-invite`	ADMIN, SUPER_ADMIN	Resend invite email (5/min)
PATCH	`/auth/users/:id`	ADMIN, SUPER_ADMIN	Update a user
DELETE	`/auth/users/:id`	ADMIN, SUPER_ADMIN	Soft-delete a user
GET	`/auth/roles`	ADMIN, SUPER_ADMIN	List all roles + permissions
POST	`/auth/roles`	ADMIN, SUPER_ADMIN	Create a custom role
PATCH	`/auth/roles/:id`	ADMIN, SUPER_ADMIN	Update role display metadata
DELETE	`/auth/roles/:id`	ADMIN, SUPER_ADMIN	Delete an unused role
PATCH	`/auth/roles/:id/permissions`	ADMIN, SUPER_ADMIN	Update the permission set for a role
GET	`/auth/users/:id/trusted-devices`	ADMIN, SUPER_ADMIN	Admin view of a user's trusted devices
DELETE	`/auth/users/:id/trusted-devices/:deviceId`	ADMIN, SUPER_ADMIN	Admin revoke of one device
DELETE	`/auth/users/:id/trusted-devices`	ADMIN, SUPER_ADMIN	Admin revoke of every device

8. User account and preferences (Me)

Controllers: `me/me.controller.ts` (`/api/me`), `preferences/preferences.controller.ts` (`/api/preferences`), `sidebar-badges/sidebar-badges.controller.ts` (`/api/me/sidebar-badges`).

Method	Path	Roles	Description
DELETE	`/me/account`	Authenticated	Soft-delete account with 30-day grace, requires current password
POST	`/me/account/cancel-deletion`	Public	Cancel pending deletion via signed emailed token
GET	`/me/export`	Authenticated	PDPL Article 13 data portability export (1/24h)
GET	`/me/consents`	Authenticated	Read caller's consent ledger + current status
POST	`/me/consents`	Authenticated	Grant or re-grant a consent
POST	`/me/consents/withdraw`	Authenticated	Withdraw a previously granted consent
PATCH	`/me/consents/marketing`	Authenticated	Convenience toggle for marketing email opt-in
GET	`/preferences`	Authenticated	Get caller's preferences (creates defaults if none)
PATCH	`/preferences/notifications`	Authenticated	Update notification preferences
PATCH	`/preferences/ui`	Authenticated	Update UI preferences
PATCH	`/preferences/policies`	MANAGER, ADMIN	Update default policy preferences
GET	`/me/sidebar-badges`	Authenticated	Unread-count map keyed by sidebar tab
POST	`/me/sidebar-badges/:tabKey/viewed`	Authenticated	Mark a sidebar tab as viewed (60/min)

9. Role requests and trainer applications

Controllers: `role-requests/role-requests.controller.ts` (`/api/role-requests`), `role-requests/support-role-requests.controller.ts` (`/api/support/role-requests`), `trainer-applications/trainer-applications.controller.ts` (`/api/trainer-applications`).

Method	Path	Roles	Description
POST	`/role-requests`	STUDENT	Student submits a role-upgrade request
GET	`/role-requests/mine`	Authenticated	Caller's own role-request history
POST	`/role-requests/blocked-national-ids`	ADMIN, SUPER_ADMIN	Block a national ID from applying
GET	`/role-requests/blocked-national-ids`	ADMIN, SUPER_ADMIN	List blocked national IDs
DELETE	`/role-requests/blocked-national-ids/:nationalId`	ADMIN, SUPER_ADMIN	Unblock a national ID
GET	`/role-requests`	ADMIN, SUPER_ADMIN	Admin queue of role requests
GET	`/role-requests/:id`	ADMIN, SUPER_ADMIN	Role-request detail
PATCH	`/role-requests/:id/approve`	ADMIN, SUPER_ADMIN	Approve the request and grant the role
PATCH	`/role-requests/:id/reject`	ADMIN, SUPER_ADMIN	Reject with reason
PATCH	`/role-requests/:id/request-info`	ADMIN, SUPER_ADMIN	Ask applicant for more information
PATCH	`/role-requests/:id`	STUDENT	Applicant resubmits after MORE_INFO
GET	`/support/role-requests`	Authenticated	Support-scoped read of role requests
GET	`/support/role-requests/:id`	Authenticated	Support-scoped role-request detail
POST	`/trainer-applications/request-otp`	Public	Captcha-protected email OTP for prospective trainer
POST	`/trainer-applications/verify-otp`	Public	Verify OTP, return short-lived scoped JWT
POST	`/trainer-applications/submit`	Public (scoped JWT)	Submit trainer application
POST	`/trainer-applications/portal/request-otp`	Public	Portal login OTP for the applicant
POST	`/trainer-applications/portal/verify-otp`	Public	Portal OTP verify, returns application snapshot
GET	`/trainer-applications/portal/status`	Public (portal JWT)	Refresh application status

Method	Path	Roles	Description
POST	`/trainer-applications/portal/resubmit`	Public (portal JWT)	Resubmit a MORE_INFO application

10. Courses

Controller: `courses/courses.controller.ts` at `/api/courses`.

Method	Path	Roles	Description
GET	`/courses/public`	Public	Active courses catalog
GET	`/courses/public/:id`	Public	Public course detail
GET	`/courses/instructors/:id/public`	Public	Public instructor profile page
GET	`/courses/featured-home`	Public	Homepage featured mix
GET	`/courses/instructor-reviews/featured`	Public	Featured instructor reviews carousel
POST	`/courses`	INSTRUCTOR, MANAGER, ADMIN	Create a new course
POST	`/courses/:id/duplicate`	INSTRUCTOR, MANAGER, ADMIN	Clone a course into a fresh draft
GET	`/courses`	INSTRUCTOR, MANAGER, ADMIN	List courses (instructor sees own)
POST	`/courses/:id/activate`	MANAGER, ADMIN	Activate a course after content review
POST	`/courses/:id/deactivate`	MANAGER, ADMIN	Archive an active course
POST	`/courses/:id/end`	MANAGER, ADMIN	Manually end an ACTIVE course
POST	`/courses/:id/complete`	MANAGER, ADMIN	Mark course as completed
POST	`/courses/:id/submit-review`	INSTRUCTOR, MANAGER, ADMIN	Submit course for manager review
POST	`/courses/:id/request-changes`	MANAGER, ADMIN	Manager requests changes
PATCH	`/courses/:id/assign-instructor`	MANAGER, ADMIN	Reassign the course instructor
GET	`/courses/:id`	INSTRUCTOR, MANAGER, ADMIN, TEACHING_ASSISTANT	Course detail with modules
PATCH	`/courses/:id`	INSTRUCTOR, MANAGER, ADMIN	Update course fields

Method	Path	Roles	Description
GET	`/courses/:id/delete-impact`	MANAGER, ADMIN	Pre-delete impact preview
DELETE	`/courses/:id`	MANAGER, ADMIN	Soft-delete a course
POST	`/courses/:id/cancel`	MANAGER, ADMIN	Amna's Phase B pre-start cancel workflow
POST	`/courses/:id/reschedule`	MANAGER, ADMIN	Phase D reschedule workflow
POST	`/courses/:id/modules`	INSTRUCTOR, MANAGER, ADMIN	Add a module
PATCH	`/courses/:id/modules/reorder`	INSTRUCTOR, MANAGER, ADMIN	Reorder modules
PATCH	`/courses/:id/modules/:mid`	INSTRUCTOR, MANAGER, ADMIN	Update a module
DELETE	`/courses/:id/modules/:mid`	INSTRUCTOR, MANAGER, ADMIN	Remove a module
POST	`/courses/:id/ratings`	STUDENT	Rate a course (one-time)
GET	`/courses/:id/ratings/me`	STUDENT	Caller's rating for a course
GET	`/courses/:id/ratings`	Public	Paginated ratings for a course
GET	`/courses/:id/ratings/stats`	Authenticated	Average rating for a course
GET	`/courses/:id/ratings/manager-view`	MANAGER, ADMIN	Ratings including moderation states
PATCH	`/courses/:id/feature`	MANAGER, ADMIN	Toggle featured flag
GET	`/courses/ratings/all`	MANAGER, ADMIN	All course rating averages (dashboard)
GET	`/courses/ratings/manager-stats`	MANAGER, ADMIN	Manager review analytics
GET	`/courses/reviews/manager`	MANAGER, ADMIN	Full moderation list

Method	Path	Roles	Description
POST	`/courses/:id/ratings/:ratingId/approve`	MANAGER, ADMIN	Approve + publish a review
POST	`/courses/:id/ratings/:ratingId/reject`	MANAGER, ADMIN	Reject + unpublish a review
PATCH	`/courses/:id/ratings/:ratingId/publish`	MANAGER, ADMIN	Toggle publication flag
DELETE	`/courses/:id/ratings/:ratingId`	MANAGER, ADMIN	Delete a rating
POST	`/courses/:id/instructor-ratings`	STUDENT	Rate the instructor for a course
GET	`/courses/:id/instructor-ratings/me`	STUDENT	Caller's instructor rating
GET	`/courses/:id/review-status`	STUDENT	Whether caller has completed the required reviews
GET	`/courses/:id/instructor-aggregated-ratings`	Public	Aggregated instructor reviews
GET	`/courses/:id/instructor-ratings`	Public	Published instructor reviews for a course
GET	`/courses/instructor-reviews/manager`	MANAGER, ADMIN	Manager instructor-review moderation queue
POST	`/courses/:id/instructor-ratings/:ratingId/approve`	MANAGER, ADMIN	Approve instructor review
POST	`/courses/:id/instructor-ratings/:ratingId/reject`	MANAGER, ADMIN	Reject instructor review
PATCH	`/courses/:id/instructor-ratings/:ratingId/publish`	MANAGER, ADMIN	Toggle instructor-review publication
DELETE	`/courses/:id/instructor-ratings/:ratingId`	MANAGER, ADMIN	Delete an instructor review

11. Sessions

Controller: `sessions/sessions.controller.ts` at `/api/sessions`.

Method	Path	Roles	Description
POST	`/sessions`	INSTRUCTOR, ADMIN, MANAGER	Schedule a new session
GET	`/sessions`	INSTRUCTOR, TA, ADMIN, MANAGER	List sessions with filters
GET	`/sessions/conflicts`	MANAGER, ADMIN	Detect scheduling conflicts
GET	`/sessions/today`	INSTRUCTOR, ADMIN, MANAGER	Today's sessions
GET	`/sessions/upcoming`	INSTRUCTOR, ADMIN, MANAGER	Upcoming sessions
GET	`/sessions/instructor-load/:instructorId`	MANAGER, ADMIN	Instructor weekly load
GET	`/sessions/instructor-schedule/:instructorId`	MANAGER, ADMIN	Instructor two-week schedule
GET	`/sessions/my-assigned`	TEACHING_ASSISTANT	TA's assigned sessions
GET	`/sessions/:id/ta-join`	TEACHING_ASSISTANT	Zoom join URL for TA with platform ID
GET	`/sessions/:id`	INSTRUCTOR, TA, ADMIN, MANAGER	Session detail
PATCH	`/sessions/:id`	INSTRUCTOR, ADMIN, MANAGER	Update a session
POST	`/sessions/:id/start`	INSTRUCTOR, ADMIN, MANAGER	Mark session live
POST	`/sessions/:id/end`	INSTRUCTOR, ADMIN, MANAGER	End session (attendance stays editable)
POST	`/sessions/:id/finalize`	INSTRUCTOR, TA, ADMIN, MANAGER	Lock attendance and mark completed
PATCH	`/sessions/:id/cancel`	INSTRUCTOR, ADMIN, MANAGER	Cancel with reason
DELETE	`/sessions/:id`	ADMIN, MANAGER	Delete completed / cancelled session
POST	`/sessions/:id/cancel-on-behalf`	MANAGER, ADMIN	Cancel on behalf of instructor
POST	`/sessions/:id/reschedule`	INSTRUCTOR, MANAGER, ADMIN	Reschedule with new date + reason

Method	Path	Roles	Description
PATCH	`/sessions/:id/assign-substitute`	MANAGER, ADMIN	Assign substitute instructor
POST	`/sessions/:id/sync-zoom`	INSTRUCTOR, TA, ADMIN, MANAGER	Manually trigger Zoom attendance sync
GET	`/sessions/:id/attendance-logs`	INSTRUCTOR, TA, ADMIN, MANAGER	Granular join/leave logs
GET	`/sessions/:id/registrants`	MANAGER, ADMIN, INSTRUCTOR	Webinar registrants list
GET	`/sessions/:id/webinar-start`	MANAGER, ADMIN, INSTRUCTOR	Zoom host start_url for a webinar
POST	`/sessions/:id/reconfigure-webinar-questions`	MANAGER, ADMIN	Re-apply standard registration questions
GET	`/sessions/:id/recordings`	INSTRUCTOR, TA, ADMIN, MANAGER, STUDENT	Session recordings
POST	`/sessions/:id/share-material`	INSTRUCTOR, ADMIN, MANAGER	Share a material with a session
DELETE	`/sessions/:id/share-material/:materialId`	INSTRUCTOR, ADMIN, MANAGER	Unshare a material
GET	`/sessions/:id/shared-materials`	Authenticated	Session's shared materials
PATCH	`/sessions/:id/assistants/:userId/role`	MANAGER, ADMIN, INSTRUCTOR	Update TA session role (co_host / assistant)

12. Enrollments and attendance

Controller: `enrollments/enrollments.controller.ts` (routes mounted under `/api/courses/*`, `/api/sessions/*`, `/api/attendance/*`).

Method	Path	Roles	Description
POST	`/courses/:id/enroll`	INSTRUCTOR, ADMIN, MANAGER	Enroll a student (with NELC limit enforcement)
GET	`/courses/:id/enrollments`	INSTRUCTOR, TA, ADMIN, MANAGER	List enrolled students
POST	`/courses/:id/manual-enroll`	MANAGER, ADMIN	Force-enroll a student with mandatory reason
DELETE	`/courses/:id/enrollments/:studentId`	MANAGER, ADMIN	Remove student (reason mandatory)
GET	`/sessions/:id/attendance`	INSTRUCTOR, TA, ADMIN, MANAGER	List attendance for a session
POST	`/sessions/:id/attendance`	INSTRUCTOR, TA, ADMIN, MANAGER	Mark single-student attendance
POST	`/sessions/:id/attendance/bulk`	INSTRUCTOR, TA, ADMIN, MANAGER	Bulk mark attendance
PATCH	`/attendance/:id`	INSTRUCTOR, TA, ADMIN, MANAGER	Update an attendance record
GET	`/courses/:id/waitlist`	INSTRUCTOR, ADMIN, MANAGER	List waitlisted students
DELETE	`/courses/:id/waitlist/:entryId`	INSTRUCTOR, ADMIN, MANAGER	Remove a student from the waitlist

13. Materials

Controller: `materials/materials.controller.ts` (routes under `/api/courses/*`, `/api/materials/*`).

Method	Path	Roles	Description
POST	`/courses/:id/materials`	INSTRUCTOR, ADMIN	Upload a material (multipart, malware-scanned)
GET	`/courses/:id/materials`	INSTRUCTOR, ADMIN, MANAGER	List materials (type / status / moduleId filters)
PATCH	`/courses/:id/modules/:moduleId/materials/reorder`	INSTRUCTOR, ADMIN	Reorder materials within a module
GET	`/materials/pending-review`	MANAGER, ADMIN	Materials awaiting review
PATCH	`/materials/:id/review`	MANAGER, ADMIN	Approve or reject a material
POST	`/materials/bulk-review`	MANAGER, ADMIN	Approve or reject many materials
GET	`/materials/:id/compliance-checklist`	MANAGER, ADMIN	5-check compliance checklist
GET	`/materials/:id/status`	INSTRUCTOR, ADMIN, MANAGER	Lightweight polling endpoint
GET	`/materials/:id/variants`	INSTRUCTOR, ADMIN, MANAGER	Transcoded video variants
PATCH	`/materials/:id`	INSTRUCTOR, ADMIN	Update material metadata
POST	`/materials/:id/replace`	INSTRUCTOR, ADMIN	Upload a new version
GET	`/materials/:id/watch-analytics`	INSTRUCTOR, ADMIN, MANAGER	Video retention analytics
GET	`/materials/:id/captions`	INSTRUCTOR, ADMIN, MANAGER	Parsed VTT captions
PUT	`/materials/:id/captions`	INSTRUCTOR, ADMIN	Save edited caption entries
POST	`/materials/:id/captions/upload`	INSTRUCTOR, ADMIN	Upload a VTT or SRT file
GET	`/materials/:id/versions`	INSTRUCTOR, ADMIN,	Version history

Method	Path	Roles	Description
		MANAGER	
POST	`/materials/:id/rollback/:versionId`	INSTRUCTOR, ADMIN	Rollback to a previous version
DELETE	`/materials/:id`	INSTRUCTOR, ADMIN, MANAGER	Remove a material

14. Assessments

Controllers: `assessments/assessments.controller.ts` (`/api/assessments`) and `assessments/assessment-extras.controller.ts` (extras mounted at root).

Method	Path	Roles	Description
POST	`/assessments`	INSTRUCTOR, ADMIN, MANAGER	Create a new assessment
GET	`/assessments`	INSTRUCTOR, TA, ADMIN, MANAGER	List assessments with filters
GET	`/assessments/course-bank/:courseId/questions`	INSTRUCTOR, ADMIN, MANAGER	Course-level question bank
POST	`/assessments/course-bank/:courseId/questions`	INSTRUCTOR, ADMIN, MANAGER	Add question to course bank
GET	`/assessments/:id`	INSTRUCTOR, ADMIN, MANAGER	Assessment detail with questions
PATCH	`/assessments/:id`	INSTRUCTOR, ADMIN, MANAGER	Update an assessment
DELETE	`/assessments/:id`	INSTRUCTOR, ADMIN, MANAGER	Delete a draft assessment
POST	`/assessments/:id/questions`	INSTRUCTOR, ADMIN, MANAGER	Add a question
GET	`/assessments/:id/questions`	INSTRUCTOR, ADMIN, MANAGER	List questions
PATCH	`/assessments/:assessmentId/questions/:questionId`	INSTRUCTOR, ADMIN, MANAGER	Update a question
DELETE	`/assessments/:assessmentId/questions/:questionId`	INSTRUCTOR, ADMIN, MANAGER	Delete a question
POST	`/assessments/:id/submit`	Authenticated	Student submits an attempt
GET	`/assessments/:id/submissions`	INSTRUCTOR, TA, ADMIN, MANAGER	List submissions
PATCH	`/assessments/:id/submissions/:submissionId/grade`	INSTRUCTOR, ADMIN, MANAGER	Grade a submission

Method	Path	Roles	Description
POST	`/assessments/:id/files`	INSTRUCTOR, ADMIN, MANAGER	Upload assignment file
DELETE	`/assessments/:id/files/:fileId`	INSTRUCTOR, ADMIN, MANAGER	Remove assignment file
PATCH	`/assessments/:id/submissions/:submissionId/grade-assignment`	INSTRUCTOR, ADMIN, MANAGER	Grade an assignment
GET	`/assessments/:id/item-analysis`	INSTRUCTOR, ADMIN, MANAGER	Difficulty and discrimination indices
GET	`/assessments/:id/difficulty-distribution`	INSTRUCTOR, ADMIN, MANAGER	Question difficulty breakdown
POST	`/assessments/:id/flag`	MANAGER, ADMIN	Flag assessment for review
POST	`/assessments/:id/override`	MANAGER, ADMIN	Grant emergency override
GET	`/assessments/:id/overrides`	MANAGER, ADMIN	List overrides
POST	`/assessments/:id/allow-retake`	MANAGER, ADMIN	Grant a retake to a student
GET	`/assessments/blueprint/:courseId`	INSTRUCTOR, ADMIN, MANAGER	Module-to-assessment coverage matrix
PATCH	`/questions/:id`	INSTRUCTOR, ADMIN	Update a question (extras)
DELETE	`/questions/:id`	INSTRUCTOR, ADMIN	Delete a question (extras)
PATCH	`/submissions/:id/grade`	INSTRUCTOR, ADMIN	Manual grade essay / short answer

15. Certificates

Controller: `certificates/certificates.controller.ts` (mixed mount).

Method	Path	Roles	Description
POST	`/certificates`	Authenticated	Student certificate request
POST	`/certificates/batch`	INSTRUCTOR, ADMIN, MANAGER	Batch process requests
GET	`/certificates/my-certificates`	Authenticated	Caller's own certificates
POST	`/certificates/batch-download`	INSTRUCTOR, ADMIN, MANAGER	Download many certificates as ZIP
GET	`/certificates/verify/:id`	Public	Public verification (30/min)
GET	`/certificates/:id/download`	Authenticated	Download an approved certificate
GET	`/certificates/:id`	INSTRUCTOR, ADMIN, MANAGER	Certificate request detail
PATCH	`/certificates/:id/approve`	INSTRUCTOR, ADMIN, MANAGER	Approve, generates PDF
PATCH	`/certificates/:id/reject`	INSTRUCTOR, ADMIN, MANAGER	Reject with note
PATCH	`/certificates/:id/hold`	INSTRUCTOR, ADMIN, MANAGER	Hold with audit note
PATCH	`/certificates/:id/revoke`	MANAGER, ADMIN	Revoke an approved certificate
GET	`/instructor/certificates`	INSTRUCTOR, ADMIN, MANAGER	Certificate requests (pending default)

16. Course and session assistants

Controllers: `course-assistants/course-assistants.controller.ts` (`/api/courses/:courseId/assistants`), `course-assistants/ta-courses.controller.ts` (`/api/course-assistants`), `session-assistants/session-assistants.controller.ts` (`/api/sessions/:sessionId/assistants`).

Method	Path	Roles	Description
GET	`/courses/:courseId/assistants/threshold-status`	INSTRUCTOR, MANAGER, ADMIN	TA threshold status
GET	`/courses/:courseId/assistants/available`	INSTRUCTOR, MANAGER, ADMIN	Available TAs
GET	`/courses/:courseId/assistants`	INSTRUCTOR, MANAGER, ADMIN	Course TAs with status
POST	`/courses/:courseId/assistants/suggest`	INSTRUCTOR	Instructor suggests a TA
POST	`/courses/:courseId/assistants`	MANAGER, ADMIN	Direct assign TA
PATCH	`/courses/:courseId/assistants/:userId/approve`	MANAGER, ADMIN	Approve suggestion
PATCH	`/courses/:courseId/assistants/:userId/reject`	MANAGER, ADMIN	Reject suggestion
DELETE	`/courses/:courseId/assistants/:userId`	MANAGER, ADMIN	Remove TA
PATCH	`/courses/:courseId/assistants/:userId/priority`	MANAGER, ADMIN	Update TA priority
GET	`/course-assistants/my-courses`	TEACHING_ASSISTANT	TA's own course assignments
GET	`/sessions/:sessionId/assistants`	INSTRUCTOR, TA, ADMIN, MANAGER	List assigned assistants
POST	`/sessions/:sessionId/assistants`	INSTRUCTOR, ADMIN, MANAGER	Assign a TA
DELETE	`/sessions/:sessionId/assistants/:userId`	INSTRUCTOR, ADMIN, MANAGER	Remove an assistant
GET	`/sessions/:sessionId/assistants/available`	INSTRUCTOR, ADMIN, MANAGER	Available TAs for the session

17. Instructor dashboard

Controller: `instructor/instructor.controller.ts` at `/api/instructor`.

Method	Path	Roles	Description
GET	`/instructor/dashboard-summary`	INSTRUCTOR, ADMIN, MANAGER	Aggregated dashboard data
GET	`/instructor/me/reviews`	INSTRUCTOR, ADMIN, MANAGER	Approved reviews for the authenticated instructor
GET	`/instructor/course-performance`	INSTRUCTOR, ADMIN, MANAGER	Per-course performance data
GET	`/instructor/roster`	MANAGER, ADMIN	Instructor roster with workload metrics
GET	`/instructor/suggestions`	MANAGER, ADMIN	Least-loaded instructor picks
GET	`/instructor/filter-options`	INSTRUCTOR, ADMIN, MANAGER	Filter dropdown options
GET	`/instructor/students`	INSTRUCTOR, ADMIN, MANAGER	All students across own courses
GET	`/instructor/at-risk`	INSTRUCTOR, ADMIN, MANAGER	At-risk learners
PATCH	`/instructor/courses/:courseId/at-risk-settings`	INSTRUCTOR, ADMIN, MANAGER	Update at-risk thresholds
POST	`/instructor/interventions`	INSTRUCTOR, ADMIN, MANAGER	Record an intervention
GET	`/instructor/students/:studentId/interventions`	INSTRUCTOR, ADMIN, MANAGER	Interventions for a student
GET	`/instructor/:id/performance`	MANAGER, ADMIN	Instructor performance KPIs
GET	`/instructor/:id/reviews`	MANAGER, ADMIN	Manager view of instructor reviews
GET	`/instructor/students/:id`	INSTRUCTOR, ADMIN, MANAGER	Student detail (enrollments, attendance, scores)

Method	Path	Roles	Description
POST	`/instructor/students/:studentId/makeup-session`	INSTRUCTOR, ADMIN, MANAGER	Create a make-up session

18. Student dashboard

Controller: `student/student.controller.ts` at `/api/student` (class-level `@Roles('STUDENT')`).

Method	Path	Description
GET	`/student/dashboard`	KPI stats
GET	`/student/enrollment-status`	Bulk enrollment check (courseIds query)
GET	`/student/courses`	Caller's enrolled courses
GET	`/student/courses/:id`	Enrolled course detail
GET	`/student/courses/:id/progress`	Detailed progress for a course
POST	`/student/courses/:id/progress`	Update lesson / module completion
POST	`/student/enroll/:id`	Self-enroll
DELETE	`/student/courses/:id/drop`	Drop a course
GET	`/student/courses/:courseId/modules/:moduleId/materials`	Materials for a module
GET	`/student/assessments/:id`	Assessment for taking
POST	`/student/assessments/:id/submit`	Submit assessment answers
POST	`/student/assessments/:id/submit-assignment`	Submit assignment files
POST	`/student/assessments/:id/save-progress`	Auto-save progress
GET	`/student/assessments/:id/progress`	Read saved progress
GET	`/student/submissions/:id/review`	Graded submission for review
GET	`/student/certificates`	Earned certificates
GET	`/student/grades/recent`	Recently graded submissions
GET	`/student/certificates/:id/download`	Download a certificate
POST	`/student/materials/:id/watch-events`	Record video watch events
GET	`/student/sessions/:id/join`	Zoom join URL with pre-filled display name
GET	`/student/calendar`	Upcoming sessions and deadlines
GET	`/student/calendar/export`	Export calendar as `.ics`
GET	`/student/recent-activity`	Recent activity timeline
GET	`/student/transcript`	Full academic transcript
GET	`/student/faqs`	FAQ categories and items
GET	`/student/tutorials`	Video tutorials
POST	`/student/tickets`	Create a support ticket
GET	`/student/tickets`	List caller's tickets

Method	Path	Description
GET	`/student/tickets/:id`	Ticket detail with replies
POST	`/student/tickets/:id/reply`	Reply to a ticket
POST	`/student/tickets/:id/rate`	Rate a resolved ticket

19. Reports

Controller: `reports/reports.controller.ts` at `/api/instructor/reports` (class-level roles INSTRUCTOR, TA, ADMIN, MANAGER).

Method	Path	Roles	Description
GET	`/instructor/reports/attendance`	class-level	Attendance report (CSV / PDF / XLSX)
GET	`/instructor/reports/gradebook`	class-level	Gradebook report
GET	`/instructor/reports/enrollment`	class-level	Enrollment report
GET	`/instructor/reports/completion`	class-level	Completion report
GET	`/instructor/reports/cme-eligibility`	class-level	CME eligibility report
GET	`/instructor/reports/webinar-registrants`	class-level	Webinar registrants
GET	`/instructor/reports/satisfaction`	class-level	Satisfaction report
GET	`/instructor/reports/instructor-performance`	class-level	Instructor performance
GET	`/instructor/reports/nelc-compliance`	class-level	NELC compliance matrix
GET	`/instructor/reports/course-catalog`	class-level	Course catalog report
GET	`/instructor/reports/revenue`	MANAGER, ADMIN	Revenue report
POST	`/instructor/reports/export-table`	class-level	Ad-hoc table export
GET	`/instructor/reports/scheduled`	class-level	Scheduled reports list
POST	`/instructor/reports/scheduled`	class-level	Create a scheduled report
PATCH	`/instructor/reports/scheduled/:id`	class-level	Update a scheduled report
DELETE	`/instructor/reports/scheduled/:id`	class-level	Delete a scheduled report

20. Payments

Controller: `payments/payments.controller.ts` at `/api/payments`.

Method	Path	Roles	Description
POST	`/payments/initiate`	STUDENT	Start a payment (10/min)
POST	`/payments/quote`	STUDENT	Read-only quote of a cart (10/min)
POST	`/payments/tabby/pre-score`	STUDENT	Tabby pre-scoring (10/min)
GET	`/payments/callback`	Public	BNPL / Moyasar redirect landing
POST	`/payments/verify/:id`	Authenticated	Manually verify a payment (20/min)
POST	`/payments/verify-charge/:chargeId`	Authenticated	Verify by gateway charge id (20/min)
GET	`/payments/gateways/health`	FINANCE, ADMIN, SUPER_ADMIN	Gateway health probe (20/min)
GET	`/payments/gateways/visible`	Public	Gateways currently visible on checkout
POST	`/payments/dev/mock-state`	Public (dev only)	Toggle payment mock state
GET	`/payments/dev/mock-state`	Public (dev only)	Read payment mock state
GET	`/payments/my-payments`	Public	Buyer-scoped list for success page
GET	`/payments`	FINANCE, ADMIN, SUPER_ADMIN	Admin all-payments list
GET	`/payments/:paymentId/detail`	FINANCE, ADMIN, SUPER_ADMIN	Payment detail
GET	`/payments/:paymentId/audit`	FINANCE, ADMIN, SUPER_ADMIN	Payment audit trail
POST	`/payments/abandoned-sweep/run`	FINANCE, ADMIN, SUPER_ADMIN	Manual abandoned-payment sweep (5/min)

21. Bank transfer

Controller: `payments/bank-transfer/bank-transfer.controller.ts` at `/api/payments/bank-transfer`.

Method	Path	Roles	Description
GET	`/payments/bank-transfer/details`	Authenticated	Bank account info shown to buyer
POST	`/payments/bank-transfer/submit-draft`	Authenticated	One-shot submit (multipart, throttled)
GET	`/payments/bank-transfer/:paymentId`	Authenticated	Public bank-transfer page hydration
POST	`/payments/bank-transfer/:paymentId/receipts`	Authenticated	Upload receipt + refund-info snapshot (throttled)
GET	`/payments/bank-transfer/:paymentId/proforma`	Authenticated	Branded proforma PDF
POST	`/payments/bank-transfer/proforma/quote`	Authenticated	Proforma from cart quote (throttled)
GET	`/payments/bank-transfer/receipts/:receiptId/download`	Owner or FINANCE	Signed URL for a receipt
GET	`/payments/bank-transfer/queue/list`	FINANCE, ADMIN, SUPER_ADMIN	Review queue list
GET	`/payments/bank-transfer/queue/summary`	FINANCE, ADMIN, SUPER_ADMIN	Queue KPI header
GET	`/payments/bank-transfer/queue/receipts/:receiptId`	FINANCE, ADMIN, SUPER_ADMIN	Full receipt review drawer
GET	`/payments/bank-transfer/queue/receipts/:receiptId/replacement-chain`	FINANCE, ADMIN, SUPER_ADMIN	Upload history for a receipt
GET	`/payments/bank-transfer/queue/receipts/:receiptId/audit`	FINANCE, ADMIN, SUPER_ADMIN	Full audit timeline
PATCH	`/payments/bank-transfer/queue/receipts/:receiptId/approve`	FINANCE, ADMIN, SUPER_ADMIN	Approve receipt
PATCH	`/payments/bank-transfer/queue/receipts/:receiptId/reject`	FINANCE, ADMIN, SUPER_ADMIN	Reject receipt (reviewerNote required)

22. Invoices

Controller: `payments/invoices/invoices.controller.ts` at `/api/invoices`.

Method	Path	Roles	Description
GET	`/invoices/verify/:invoiceNumber`	Public	Public invoice verification
GET	`/invoices/my-invoices`	STUDENT	Caller's invoices
GET	`/invoices/summary`	FINANCE, ADMIN, SUPER_ADMIN	Invoice KPI summary
GET	`/invoices/export.csv`	FINANCE, ADMIN, SUPER_ADMIN	CSV export
POST	`/invoices/overdue-sweep/run`	FINANCE, ADMIN, SUPER_ADMIN	Manual overdue sweep
GET	`/invoices/:id/pdf`	STUDENT, FINANCE, ADMIN, SUPER_ADMIN	Invoice PDF
GET	`/invoices/:id`	STUDENT, FINANCE, ADMIN, SUPER_ADMIN	Invoice detail
GET	`/invoices`	FINANCE, ADMIN, SUPER_ADMIN	All invoices list
GET	`/invoices/:id/detail`	FINANCE, ADMIN, SUPER_ADMIN	Admin invoice detail
GET	`/invoices/:id/audit`	FINANCE, ADMIN, SUPER_ADMIN	Invoice audit trail
POST	`/invoices/:id/regenerate-pdf`	FINANCE, ADMIN, SUPER_ADMIN	Re-render invoice PDF
POST	`/invoices/:id/cancel`	FINANCE, ADMIN, SUPER_ADMIN	Cancel invoice
POST	`/invoices/:id/resend`	FINANCE, ADMIN, SUPER_ADMIN	Resend invoice email

23. Refunds

Controller: `payments/refunds/refunds.controller.ts` at `/api/refunds`.

Method	Path	Roles	Description
POST	`/refunds`	STUDENT, FINANCE, ADMIN, SUPER_ADMIN	Request a refund
POST	`/refunds/admin-initiate`	FINANCE, ADMIN, SUPER_ADMIN	Admin-initiated refund
GET	`/refunds/summary`	FINANCE, ADMIN, SUPER_ADMIN	Refund KPI summary
GET	`/refunds/refundable-payments`	FINANCE, ADMIN, SUPER_ADMIN	Eligible payments picker
GET	`/refunds/preview/:paymentId`	FINANCE, ADMIN, SUPER_ADMIN	Refund preview
GET	`/refunds/my-refunds`	STUDENT	Caller's refunds
GET	`/refunds/export`	FINANCE, ADMIN, SUPER_ADMIN	Refunds export
GET	`/refunds`	FINANCE, ADMIN, SUPER_ADMIN	All refunds
GET	`/refunds/:id/detail`	FINANCE, ADMIN, SUPER_ADMIN	Refund detail
GET	`/refunds/:id/audit`	FINANCE, ADMIN, SUPER_ADMIN	Refund audit
GET	`/refunds/:id`	FINANCE, ADMIN, SUPER_ADMIN	Refund by id
PATCH	`/refunds/:id/approve`	FINANCE, ADMIN, SUPER_ADMIN	Approve refund
PATCH	`/refunds/:id/reject`	FINANCE, ADMIN, SUPER_ADMIN	Reject refund
PATCH	`/refunds/:id/mark-failed`	FINANCE, ADMIN, SUPER_ADMIN	Mark as failed

24. Discounts and campaigns

Controller: `payments/discounts/discounts.controller.ts` at `/api/discounts`.

Method	Path	Roles	Description
POST	`/discounts/validate`	STUDENT	Validate a discount code against a cart
GET	`/discounts/course-sales`	FINANCE, ADMIN, SUPER_ADMIN	Per-course sale list
PUT	`/discounts/course-sales/:courseId`	FINANCE, ADMIN, SUPER_ADMIN	Upsert a per-course sale
GET	`/discounts/analytics`	FINANCE, ADMIN, SUPER_ADMIN	Discount analytics
GET	`/discounts/summary`	FINANCE, ADMIN, SUPER_ADMIN	Summary tiles
GET	`/discounts/chart-series`	FINANCE, ADMIN, SUPER_ADMIN	Chart data
GET	`/discounts/activity/recent`	FINANCE, ADMIN, SUPER_ADMIN	Recent activity
GET	`/discounts/early-bird`	FINANCE, ADMIN, SUPER_ADMIN	Early-bird promotions
GET	`/discounts/batches`	FINANCE, ADMIN, SUPER_ADMIN	Batch-issued code sets
POST	`/discounts/auto-apply`	Public	Best auto-apply discount for a cart
GET	`/discounts/active-banner-campaign`	Public	Active banner campaigns
GET	`/discounts/campaigns/report`	Public	Campaign performance report
GET	`/discounts/campaigns`	FINANCE, ADMIN, SUPER_ADMIN	List campaigns
POST	`/discounts/campaigns`	FINANCE, ADMIN, SUPER_ADMIN	Create campaign
GET	`/discounts/campaigns/:id`	FINANCE, ADMIN, SUPER_ADMIN	Campaign detail
PATCH	`/discounts/campaigns/:id`	FINANCE, ADMIN, SUPER_ADMIN	Update campaign
DELETE	`/discounts/campaigns/:id`	FINANCE, ADMIN, SUPER_ADMIN	Delete campaign
GET	`/discounts/export`	FINANCE, ADMIN, SUPER_ADMIN	Discounts CSV export
GET	`/discounts`	FINANCE, ADMIN, SUPER_ADMIN	List discounts

Method	Path	Roles	Description
POST	`/discounts`	FINANCE, ADMIN, SUPER_ADMIN	Create discount
POST	`/discounts/bulk`	FINANCE, ADMIN, SUPER_ADMIN	Bulk-generate codes
GET	`/discounts/:id/audit`	FINANCE, ADMIN, SUPER_ADMIN	Discount audit trail
GET	`/discounts/:id`	FINANCE, ADMIN, SUPER_ADMIN	Discount detail
PATCH	`/discounts/:id`	FINANCE, ADMIN, SUPER_ADMIN	Update a discount
DELETE	`/discounts/:id`	FINANCE, ADMIN, SUPER_ADMIN	Delete a discount

25. Reconciliation and escalations

Controllers: `payments/reconciliation/reconciliation.controller.ts` (`/api/payments/reconciliation` , class-level FINANCE / ADMIN / SUPER_ADMIN), `payments/reconciliation/escalations.controller.ts` , `payments/reconciliation/reconciliation-cron.controller.ts` (public cron endpoints).

Method	Path	Roles	Description
GET	`/payments/reconciliation/stats`	FINANCE, ADMIN, SUPER_ADMIN	Reconciliation KPI stats
GET	`/payments/reconciliation/findings`	FINANCE, ADMIN, SUPER_ADMIN	Findings list with filters
GET	`/payments/reconciliation/findings/export`	FINANCE, ADMIN, SUPER_ADMIN	Findings export (CSV / XLSX / PDF)
POST	`/payments/reconciliation/scan`	FINANCE, ADMIN, SUPER_ADMIN	Trigger scan on demand (5/min)
PATCH	`/payments/reconciliation/findings/:id/resolve`	FINANCE, ADMIN, SUPER_ADMIN	Resolve one finding
PATCH	`/payments/reconciliation/findings/:id/ignore`	FINANCE, ADMIN, SUPER_ADMIN	Ignore one finding
PATCH	`/payments/reconciliation/findings/bulk-resolve`	FINANCE, ADMIN, SUPER_ADMIN	Bulk resolve (throttled)
PATCH	`/payments/reconciliation/findings/bulk-ignore`	FINANCE, ADMIN, SUPER_ADMIN	Bulk ignore (throttled)
POST	`/payments/reconciliation/findings/:findingId/escalate`	FINANCE, ADMIN, SUPER_ADMIN	Create escalation from a finding
GET	`/payments/reconciliation/escalations`	FINANCE, ADMIN, SUPER_ADMIN	List escalations
GET	`/payments/reconciliation/escalations/stats`	FINANCE, ADMIN, SUPER_ADMIN	Escalation stats
GET	`/payments/reconciliation/escalations/:id`	FINANCE, ADMIN, SUPER_ADMIN	Escalation detail
PATCH	`/payments/reconciliation/escalations/:id/claim`	ADMIN, SUPER_ADMIN	Claim ownership
PATCH	`/payments/reconciliation/escalations/:id/send-back`	ADMIN, SUPER_ADMIN	Send back to finance

Method	Path	Roles	Description
PATCH	`/payments/reconciliation/escalations/:id/close`	ADMIN, SUPER_ADMIN	Close escalation
POST	`/payments/reconciliation/escalations/:id/enroll`	ADMIN, SUPER_ADMIN	Manual enrollment from escalation (throttled)
POST	`/payments/reconciliation/escalations/:id/refund`	ADMIN, SUPER_ADMIN	Manual refund from escalation (throttled)
POST	`/payments/reconciliation/cron/daily-scan`	Public (OIDC / token)	Cloud Scheduler daily scan
POST	`/payments/reconciliation/cron/stale-reminders`	Public (OIDC / token)	Cloud Scheduler stale reminders (5/min)
POST	`/payments/reconciliation/cron/abandoned-sweep`	Public (OIDC / token)	Cloud Scheduler abandoned sweep (5/min)
POST	`/payments/reconciliation/cron/overdue-invoices`	Public (OIDC / token)	Cloud Scheduler overdue invoices (5/min)
POST	`/payments/reconciliation/cron/bnpl-status-poll`	Public (OIDC / token)	Cloud Scheduler BNPL status poll

26. Finance dashboard and settings

Controllers: `payments/finance/finance.controller.ts` (`/api/finance`), `payments/finance/finance-settings.controller.ts` (`/api/finance/settings`), `payments/finance/public-tax.controller.ts` (`/api/public/tax`).

Method	Path	Roles	Description
POST	`/finance/tamara/retry/:paymentId`	FINANCE, ADMIN, SUPER_ADMIN	Retry Tamara authorise + capture
POST	`/finance/tamara/cancel/:paymentId`	FINANCE, ADMIN, SUPER_ADMIN	Cancel a pre-capture Tamara order
GET	`/finance/summary`	FINANCE, ADMIN, SUPER_ADMIN	Finance KPI summary
GET	`/finance/revenue-by-source`	FINANCE, ADMIN, SUPER_ADMIN	Revenue by source
GET	`/finance/payment-status`	FINANCE, ADMIN, SUPER_ADMIN	Payment status donut
GET	`/finance/refund-rate`	FINANCE, ADMIN, SUPER_ADMIN	Refund rate KPI
GET	`/finance/invoice-aging`	FINANCE, ADMIN, SUPER_ADMIN	Invoice aging bands
GET	`/finance/bank-transfer-review-aging`	FINANCE, ADMIN, SUPER_ADMIN	Bank-transfer review aging
GET	`/finance/revenue-trend`	FINANCE, ADMIN, SUPER_ADMIN	Revenue trend series
GET	`/finance/top-courses`	FINANCE, ADMIN, SUPER_ADMIN	Top courses by revenue
GET	`/finance/revenue-by-course`	FINANCE, ADMIN, SUPER_ADMIN	Revenue by course
GET	`/finance/revenue-by-instructor`	FINANCE, ADMIN, SUPER_ADMIN	Revenue by instructor
GET	`/finance/revenue-by-delivery-type`	FINANCE, ADMIN, SUPER_ADMIN	Revenue by delivery type
GET	`/finance/chargeback-rate`	FINANCE, ADMIN, SUPER_ADMIN	Chargeback rate KPI
GET	`/finance/export`	FINANCE, ADMIN, SUPER_ADMIN	Payments export
GET	`/finance/reports/revenue`	FINANCE, ADMIN, SUPER_ADMIN	Revenue report preview
GET	`/finance/reports/refunds`	FINANCE, ADMIN, SUPER_ADMIN	Refunds report preview
GET	`/finance/reports/payments`	FINANCE, ADMIN, SUPER_ADMIN	Payments report preview
GET	`/finance/reports/course-sales`	FINANCE, ADMIN, SUPER_ADMIN	Course sales preview

Method	Path	Roles	Description
GET	`/finance/reports/webinar-sales`	FINANCE, ADMIN, SUPER_ADMIN	Webinar sales preview
GET	`/finance/reports/student-purchases`	FINANCE, ADMIN, SUPER_ADMIN	Student purchases preview
GET	`/finance/reports/finance-activity`	FINANCE, ADMIN, SUPER_ADMIN	Finance activity preview
GET	`/finance/reports/vat`	FINANCE, ADMIN, SUPER_ADMIN	VAT report preview
GET	`/finance/settings/gateways`	FINANCE, ADMIN, SUPER_ADMIN	Gateway settings snapshot
GET	`/finance/settings/refund-policy`	FINANCE, ADMIN, SUPER_ADMIN	Refund policy settings
PUT	`/finance/settings/refund-policy`	FINANCE, ADMIN, SUPER_ADMIN	Update refund policy
GET	`/finance/settings/tax`	FINANCE, ADMIN, SUPER_ADMIN	Tax settings
PUT	`/finance/settings/tax`	FINANCE, ADMIN, SUPER_ADMIN	Update tax settings
GET	`/finance/settings/vat-treatments`	FINANCE, ADMIN, SUPER_ADMIN	Per-course VAT treatment
PUT	`/finance/settings/vat-treatments/:courseId`	FINANCE, ADMIN, SUPER_ADMIN	Update one course's VAT treatment
POST	`/finance/settings/vat-treatments/bulk`	FINANCE, ADMIN, SUPER_ADMIN	Bulk update VAT treatments
GET	`/public/tax/display`	Public	Public tax display mode

27. Student finance area

Controller: `payments/student/student-finance.controller.ts` at `/api/student/finance` (class-level `@Roles('STUDENT')`).

Method	Path	Description
GET	`/student/finance/overview`	Landing tiles + lifetime / YTD totals + last 5 payments
GET	`/student/finance/payments`	Caller's payment history with filters
GET	`/student/finance/payments/:id`	Single payment detail + eligibility
GET	`/student/finance/payment-by-enrollment/:enrollmentId`	Resolve enrollment id to payment id
GET	`/student/finance/policy-snapshot`	Refund eligibility + policy snapshot

28. Admin finance actions

Controller: `payments/admin/admin-finance.controller.ts` at `/api/admin/finance` (class-level ADMIN / SUPER_ADMIN).

Method	Path	Description
GET	`/admin/finance/gateways/runtime-status`	Per-gateway mode / source / lastFour pills
POST	`/admin/finance/gateways/:gateway/test`	Test a gateway credential set
POST	`/admin/finance/gateways/refresh-cache`	Refresh cached gateway credentials
POST	`/admin/finance/payments/:id/force-status`	Manually flip payment to PAID or FAILED
POST	`/admin/finance/payments/:id/replay-webhook`	Replay the success path
POST	`/admin/finance/payments/:id/sync-from-gateway`	Poll gateway for live status and apply
POST	`/admin/finance/invoices/:id/force-cancel`	Cancel invoice even when paid
POST	`/admin/finance/invoices/:id/resend-email`	Resend invoice + confirmation email
POST	`/admin/finance/invoices/:id/regenerate-pdf`	Re-render an invoice PDF
POST	`/admin/finance/refunds/:id/force-approve`	Approve refund even when policy blocks

Payment gateway webhooks are covered in section 43.

29. Support (tickets, KB, CSAT, SLA)

Controller: `support/support.controller.ts` at `/api/tickets`.

29.1 Ticket lifecycle

Method	Path	Roles	Description
POST	`/tickets`	Authenticated	Create a ticket
GET	`/tickets`	Authenticated	List tickets (support / admin see all)
GET	`/tickets/my-tickets`	Authenticated	Caller's own tickets
GET	`/tickets/looped-in`	Authenticated	Tickets the caller is a collaborator on
GET	`/tickets/:id`	Authenticated	Ticket detail
PATCH	`/tickets/:id`	SUPPORT, ADMIN, SUPER_ADMIN	Update ticket
POST	`/tickets/:id/comments`	Authenticated	Add a comment
POST	`/tickets/:id/hold-requests`	SUPPORT	Create hold request
PATCH	`/tickets/:id/hold-requests/:requestId`	SUPPORT, ADMIN, SUPER_ADMIN	Decide hold request
POST	`/tickets/:id/escalate`	SUPPORT, ADMIN, SUPER_ADMIN	Escalate ticket
POST	`/tickets/:id/resolve`	SUPPORT, ADMIN, SUPER_ADMIN	Resolve ticket
POST	`/tickets/:id/close`	SUPPORT, ADMIN, SUPER_ADMIN	Close ticket
POST	`/tickets/:id/apply-macro`	SUPPORT, ADMIN, SUPER_ADMIN	Apply macro to ticket
POST	`/tickets/:id/rate`	STUDENT	Rate a resolved ticket
POST	`/tickets/:id/merge`	SUPPORT, ADMIN, SUPER_ADMIN	Merge two tickets
POST	`/tickets/:id/transfer`	SUPPORT, ADMIN, SUPER_ADMIN	Transfer to another department
GET	`/tickets/:id/collaborators`	Authenticated	List collaborators
POST	`/tickets/:id/collaborators`	SUPPORT, ADMIN, SUPER_ADMIN	Add collaborator
DELETE	`/tickets/:id/collaborators/:userId`	SUPPORT, ADMIN, SUPER_ADMIN	Remove collaborator
GET	`/tickets/:id/export`	Authenticated	Export ticket history
GET	`/tickets/:id/export-archive`	Authenticated	Export full archive

Method	Path	Roles	Description
GET	`/tickets/:id/merge-candidates`	SUPPORT, ADMIN, SUPER_ADMIN	Suggested merge targets
GET	`/tickets/:id/student-context`	Authenticated	Student context panel
GET	`/tickets/:id/student-timeline`	Authenticated	Student activity timeline
GET	`/tickets/:id/certificate-details`	Authenticated	Certificate context for ticket
GET	`/tickets/:id/trainer-application`	Authenticated	Trainer-application context
GET	`/tickets/:id/audit`	SUPPORT, ADMIN, SUPER_ADMIN	Ticket audit log
POST	`/tickets/:id/attachment-download`	Authenticated	Record attachment download event
GET	`/tickets/:id/attachment-download-log`	SUPPORT, ADMIN, SUPER_ADMIN	Attachment download log
GET	`/tickets/:id/attachment-history`	SUPPORT, ADMIN, SUPER_ADMIN	Attachment version history
DELETE	`/tickets/:id/attachments/:attachmentId`	SUPPORT, ADMIN, SUPER_ADMIN	Soft-delete attachment
POST	`/tickets/on-behalf`	INSTRUCTOR, ADMIN, SUPER_ADMIN	Instructor creates ticket for learner
GET	`/tickets/opened-by-me`	INSTRUCTOR, ADMIN, SUPER_ADMIN	Tickets opened by caller
GET	`/tickets/course/:courseId/count`	Authenticated	Course-scoped ticket count
PATCH	`/tickets/:id/tags`	SUPPORT, ADMIN, SUPER_ADMIN	Set ticket tags

29.2 KPIs, reports, and CSAT

Method	Path	Roles	Description
GET	`/tickets/kpis`	SUPPORT, ADMIN, SUPER_ADMIN	Support KPIs
GET	`/tickets/activity`	SUPPORT, ADMIN, SUPER_ADMIN	Activity feed
GET	`/tickets/agents`	SUPPORT, ADMIN, SUPER_ADMIN	Agent directory
GET	`/tickets/sla/breaches`	SUPPORT, ADMIN, SUPER_ADMIN	SLA breach list
GET	`/tickets/sla/backlog`	SUPPORT, ADMIN, SUPER_ADMIN	SLA backlog
POST	`/tickets/agents`	SUPPORT, ADMIN, SUPER_ADMIN	Create support agent
GET	`/tickets/reports/trend`	SUPPORT, ADMIN, SUPER_ADMIN	Ticket volume trend
GET	`/tickets/reports/csat-comments`	SUPPORT, ADMIN, SUPER_ADMIN	Recent CSAT comments
GET	`/tickets/reports/volume`	SUPPORT, ADMIN, SUPER_ADMIN	Volume report
GET	`/tickets/reports/issues`	SUPPORT, ADMIN, SUPER_ADMIN	Top issue categories
GET	`/tickets/reports/sla-compliance`	SUPPORT, ADMIN, SUPER_ADMIN	SLA compliance report
GET	`/tickets/reports/agent-performance`	SUPPORT, ADMIN, SUPER_ADMIN	Agent performance
GET	`/tickets/reports/export`	SUPPORT, ADMIN, SUPER_ADMIN	Report CSV export
GET	`/tickets/reports/export-pdf`	SUPPORT, ADMIN, SUPER_ADMIN	Report PDF export
GET	`/tickets/csat/summary`	SUPPORT, ADMIN, SUPER_ADMIN	CSAT summary
GET	`/tickets/csat/by-category`	SUPPORT, ADMIN, SUPER_ADMIN	CSAT by category
GET	`/tickets/csat/by-agent`	SUPPORT, ADMIN, SUPER_ADMIN	CSAT by agent
GET	`/tickets/csat/monthly`	SUPPORT, ADMIN, SUPER_ADMIN	CSAT monthly

Method	Path	Roles	Description
GET	`/tickets/csat/low-ratings`	SUPPORT, ADMIN, SUPER_ADMIN	Low-rated surveys
POST	`/tickets/csat/:surveyId/follow-up`	SUPPORT, ADMIN, SUPER_ADMIN	Mark CSAT survey followed up
GET	`/tickets/nps/summary`	SUPPORT, ADMIN, SUPER_ADMIN	NPS summary
GET	`/tickets/csat/pending`	Authenticated	Pending CSAT for caller
GET	`/tickets/csat/:surveyId`	Public	Retrieve CSAT survey (10/min)
POST	`/tickets/csat/:surveyId`	Public	Submit CSAT (5/min)

29.3 Team, orphans, supervisors

Method	Path	Roles	Description
GET	`/tickets/team/my`	SUPPORT, ADMIN, SUPER_ADMIN	Caller's team
GET	`/tickets/team/available-agents`	SUPPORT, ADMIN, SUPER_ADMIN	Available agents
GET	`/tickets/team/members/:agentId`	SUPPORT, ADMIN, SUPER_ADMIN	Team member detail
DELETE	`/tickets/team/members/:agentId`	SUPPORT, ADMIN, SUPER_ADMIN	Remove team member
POST	`/tickets/team/requests`	SUPPORT, ADMIN, SUPER_ADMIN	Create team change request
GET	`/tickets/team/requests/mine`	SUPPORT, ADMIN, SUPER_ADMIN	Caller's team requests
GET	`/tickets/team/requests`	ADMIN, SUPER_ADMIN	All team requests
GET	`/tickets/team/requests/pending-count`	ADMIN, SUPER_ADMIN	Pending team-request count
POST	`/tickets/team/requests/:id/decision`	ADMIN, SUPER_ADMIN	Decide a team request
GET	`/tickets/team/active-supervisors`	ADMIN, SUPER_ADMIN	Active supervisors picker
GET	`/tickets/team/orphans`	ADMIN, SUPER_ADMIN	Orphan agents
POST	`/tickets/team/orphans/bulk-assign`	ADMIN, SUPER_ADMIN	Bulk-assign orphan agents
POST	`/tickets/team/reassign`	ADMIN, SUPER_ADMIN	Reassign supervisor's team
GET	`/tickets/admin/support-kpis`	ADMIN, SUPER_ADMIN	Admin support KPIs
GET	`/tickets/admin/support-overview`	ADMIN, SUPER_ADMIN	Admin support overview
GET	`/tickets/admin/supervisors`	ADMIN, SUPER_ADMIN	Supervisors with stats
GET	`/tickets/admin/supervisors/:id`	ADMIN, SUPER_ADMIN	Supervisor detail
GET	`/tickets/admin/audit-trail`	ADMIN, SUPER_ADMIN	Support audit trail
GET	`/tickets/audit-trail`	SUPPORT, ADMIN, SUPER_ADMIN	Scoped support audit trail
GET	`/tickets/my-performance`	SUPPORT, ADMIN, SUPER_ADMIN	Caller performance

29.4 Settings, SLA, macros, tags, KB

Method	Path	Roles	Description
GET	`/tickets/settings/business-hours`	Public	Business hours
PATCH	`/tickets/settings/business-hours`	ADMIN, SUPER_ADMIN	Update business hours
GET	`/tickets/settings/auto-assignment`	SUPPORT, ADMIN, SUPER_ADMIN	Auto-assignment settings
PATCH	`/tickets/settings/auto-assignment`	SUPPORT, ADMIN, SUPER_ADMIN	Update auto-assignment
GET	`/tickets/settings/ticket-categories`	Public	Ticket category list
POST	`/tickets/settings/ticket-categories`	SUPPORT, ADMIN, SUPER_ADMIN	Add category
PATCH	`/tickets/settings/ticket-categories/:slug`	SUPPORT, ADMIN, SUPER_ADMIN	Update category
DELETE	`/tickets/settings/ticket-categories/:slug`	SUPPORT, ADMIN, SUPER_ADMIN	Remove category
GET	`/tickets/settings/notification-prefs`	Authenticated	Notification prefs
PATCH	`/tickets/settings/notification-prefs`	Authenticated	Update notification prefs
GET	`/tickets/sla-policies`	SUPPORT, ADMIN, SUPER_ADMIN	SLA policies
PUT	`/tickets/sla-policies/:priority`	SUPPORT, ADMIN, SUPER_ADMIN	Update SLA policy
GET	`/tickets/config/retention`	SUPPORT, ADMIN, SUPER_ADMIN	Retention config
PATCH	`/tickets/config/retention`	SUPPORT, ADMIN, SUPER_ADMIN	Update retention
GET	`/tickets/macros`	SUPPORT, ADMIN, SUPER_ADMIN	Macros list
GET	`/tickets/macros/usage-stats`	SUPPORT, ADMIN, SUPER_ADMIN	Macro usage stats
GET	`/tickets/macros/:id`	SUPPORT, ADMIN, SUPER_ADMIN	Macro detail
POST	`/tickets/macros`	SUPPORT, ADMIN, SUPER_ADMIN	Create macro
PATCH	`/tickets/macros/:id`	SUPPORT, ADMIN, SUPER_ADMIN	Update macro

Method	Path	Roles	Description
DELETE	`/tickets/macros/:id`	SUPPORT, ADMIN, SUPER_ADMIN	Delete macro
GET	`/tickets/tags`	SUPPORT, ADMIN, SUPER_ADMIN	List tags
POST	`/tickets/tags`	SUPPORT, ADMIN, SUPER_ADMIN	Create tag
PATCH	`/tickets/tags/:id`	SUPPORT, ADMIN, SUPER_ADMIN	Update tag
DELETE	`/tickets/tags/:id`	SUPPORT, ADMIN, SUPER_ADMIN	Delete tag
POST	`/tickets/tags/:id/archive`	SUPPORT, ADMIN, SUPER_ADMIN	Archive tag
POST	`/tickets/tags/:id/unarchive`	SUPPORT, ADMIN, SUPER_ADMIN	Unarchive tag
GET	`/tickets/tags/:id/rules`	SUPPORT, ADMIN, SUPER_ADMIN	Tag auto-apply rules
POST	`/tickets/tags/:id/rules`	SUPPORT, ADMIN, SUPER_ADMIN	Create tag rule
PATCH	`/tickets/tag-rules/:ruleId`	SUPPORT, ADMIN, SUPER_ADMIN	Update tag rule
DELETE	`/tickets/tag-rules/:ruleId`	SUPPORT, ADMIN, SUPER_ADMIN	Delete tag rule
GET	`/tickets/tags/analytics/summary`	SUPPORT, ADMIN, SUPER_ADMIN	Tag analytics
GET	`/tickets/departments/:dept/members`	SUPPORT, ADMIN, SUPER_ADMIN	Department members picker
GET	`/tickets/kb/categories`	Public	Help Center categories
POST	`/tickets/kb/categories`	SUPPORT, ADMIN, SUPER_ADMIN	Create KB category
PATCH	`/tickets/kb/categories/:id`	SUPPORT, ADMIN, SUPER_ADMIN	Update KB category
DELETE	`/tickets/kb/categories/:id`	SUPPORT, ADMIN, SUPER_ADMIN	Delete KB category
GET	`/tickets/kb/articles`	Public	Help Center articles list
GET	`/tickets/kb/articles/:id`	Public	Help Center article detail

Method	Path	Roles	Description
POST	`/tickets/kb/articles`	SUPPORT, ADMIN, SUPER_ADMIN	Create KB article
PATCH	`/tickets/kb/articles/:id`	SUPPORT, ADMIN, SUPER_ADMIN	Update KB article
DELETE	`/tickets/kb/articles/:id`	SUPPORT, ADMIN, SUPER_ADMIN	Delete KB article
POST	`/tickets/kb/searches`	Public	Record KB search analytics (60/min)
GET	`/tickets/kb/search-analytics`	SUPPORT, ADMIN, SUPER_ADMIN	KB search analytics
GET	`/tickets/kb/articles/analytics/most-viewed`	SUPPORT, ADMIN, SUPER_ADMIN	Most-viewed KB articles
GET	`/tickets/kb/articles/analytics/low-engagement`	SUPPORT, ADMIN, SUPER_ADMIN	Low-engagement KB articles

30. Notifications and badges

Controllers: `notifications/notifications.controller.ts` (`/api/notifications`), `user-notifications/user-notifications.controller.ts` (`/api/user-notifications`), `admin-notifications/admin-notifications.controller.ts` (`/api/admin/notifications`).

Method	Path	Roles	Description
GET	`/notifications`	ADMIN, SUPER_ADMIN	Admin notifications list
PATCH	`/notifications/read-all`	ADMIN, SUPER_ADMIN	Mark all as read
PATCH	`/notifications/:id/read`	ADMIN, SUPER_ADMIN	Mark one as read
GET	`/user-notifications`	Authenticated	Notifications list + unread count
GET	`/user-notifications/unread-count`	Authenticated	Lightweight polling endpoint
PATCH	`/user-notifications/:id/read`	Authenticated	Mark one as read
PATCH	`/user-notifications/read-all`	Authenticated	Mark all as read
DELETE	`/user-notifications/clear-all`	Authenticated	Delete all notifications for caller
GET	`/admin/notifications`	ADMIN, SUPER_ADMIN	Admin ops notifications
GET	`/admin/notifications/counts`	ADMIN, SUPER_ADMIN	Admin ops notification counts
PATCH	`/admin/notifications/:id/read`	ADMIN, SUPER_ADMIN	Mark admin notification read
POST	`/admin/notifications/mark-all-read`	ADMIN, SUPER_ADMIN	Mark all admin notifications read

31. Points and rewards

Controllers: `points/points.controller.ts` (`/api/points/me`), `points/points-admin.controller.ts` (`/api/admin/points`).

Method	Path	Roles	Description
GET	`/points/me/balance`	Authenticated	Current points balance
GET	`/points/me`	Authenticated	Balance + earnedThisMonth
GET	`/points/me/rules`	Authenticated	Active earn rules
GET	`/points/me/transactions`	Authenticated	Ledger transactions
POST	`/points/me/redeem-course/:courseId`	Authenticated	Redeem points to unlock a course (path variant)
POST	`/points/me/redeem-course`	Authenticated	Redeem points (body variant)
GET	`/points/me/redeemable-courses`	Authenticated	Courses caller can redeem
GET	`/points/me/rewards`	Authenticated	Redeemable rewards catalog
GET	`/points/me/rewards/redemptions`	Authenticated	Caller's redemption receipts
GET	`/points/me/rewards/history`	Authenticated	Paginated My-Rewards history
POST	`/points/me/rewards/:id/redeem`	Authenticated	Redeem a reward
GET	`/admin/points/rules`	ADMIN, SUPER_ADMIN	Points earn rules
PATCH	`/admin/points/rules/:ruleKey`	ADMIN, SUPER_ADMIN	Update a rule
GET	`/admin/points/config`	ADMIN, SUPER_ADMIN	Points config singleton
PATCH	`/admin/points/config`	ADMIN, SUPER_ADMIN	Update points config
POST	`/admin/points/grant`	ADMIN, SUPER_ADMIN	Manual grant / deduction
GET	`/admin/points/courses`	ADMIN, SUPER_ADMIN	Courses with redemption settings
DELETE	`/admin/points/courses/:courseId`	ADMIN, SUPER_ADMIN	Remove per-course redemption row
POST	`/admin/points/courses/bulk-enable`	ADMIN, SUPER_ADMIN	Bulk-enable redemption for paid courses
GET	`/admin/points/webinar-sessions`	ADMIN, SUPER_ADMIN	Webinar sessions for free_webinar reward

Method	Path	Roles	Description
PATCH	`/admin/points/courses/:courseId`	ADMIN, SUPER_ADMIN	Upsert per-course redemption settings
GET	`/admin/points/users/search`	ADMIN, SUPER_ADMIN	Search students for admin points tabs
GET	`/admin/points/users/:userId/transactions`	ADMIN, SUPER_ADMIN	User's ledger
GET	`/admin/points/users/:userId/balance`	ADMIN, SUPER_ADMIN	User's balance
GET	`/admin/points/rewards`	ADMIN, SUPER_ADMIN	Rewards catalog CRUD
POST	`/admin/points/rewards`	ADMIN, SUPER_ADMIN	Create reward
PATCH	`/admin/points/rewards/:id`	ADMIN, SUPER_ADMIN	Update reward
DELETE	`/admin/points/rewards/:id`	ADMIN, SUPER_ADMIN	Delete reward
GET	`/admin/points/redemptions`	ADMIN, SUPER_ADMIN	Redemptions list
POST	`/admin/points/redemptions/:id/transition`	ADMIN, SUPER_ADMIN	Transition redemption state
GET	`/admin/points/course-redemptions`	ADMIN, SUPER_ADMIN	Course-redemption analytics
GET	`/admin/points/course-redemptions/export.csv`	ADMIN, SUPER_ADMIN	CSV export

32. PDPL admin

Controller: `pdpl/pdpl-admin.controller.ts` at `/api/admin/pdpl` (class-level ADMIN / SUPER_ADMIN).

Method	Path	Description
GET	`/admin/pdpl/requests`	List PDPL requests
GET	`/admin/pdpl/summary`	PDPL summary tiles

33. NELC compliance, audit, and xAPI

Controllers: `nelc-compliance/nelc-compliance.controller.ts` (`/api/nelc-compliance`), `nelc-audit/nelc-audit.controller.ts` (`/api/nelc-audit`), `xapi/xapi.admin.controller.ts` (`/api/admin/integrations/nelc` , class-level MANAGER / ADMIN / SUPER_ADMIN).

Method	Path	Roles	Description
GET	`/nelc-compliance/summary`	Authenticated	Compliance summary
GET	`/nelc-compliance/all-courses`	Authenticated	Per-course compliance
GET	`/nelc-compliance/export.csv`	Authenticated	CSV export
GET	`/nelc-compliance/session/:id`	Authenticated	Session compliance
GET	`/nelc-compliance/course/:id`	Authenticated	Course compliance
GET	`/nelc-compliance/course/:id/scorecard`	Authenticated	Course scorecard
GET	`/nelc-compliance/course/:id/evidence`	Authenticated	Evidence bundle download
GET	`/nelc-compliance/virtual-classes`	Authenticated	Virtual-class compliance
GET	`/nelc-audit`	ADMIN, SUPER_ADMIN	Paginated audit logs
GET	`/nelc-audit/summary`	ADMIN, SUPER_ADMIN	Aggregated counts by event type
GET	`/nelc-audit/export`	ADMIN, SUPER_ADMIN, MANAGER	CSV export
GET	`/admin/integrations/nelc/mode`	MANAGER, ADMIN, SUPER_ADMIN	Active LRS mode
POST	`/admin/integrations/nelc/mode`	SUPER_ADMIN	Switch LRS environment
GET	`/admin/integrations/nelc/config`	MANAGER, ADMIN, SUPER_ADMIN	Sanitised config view
GET	`/admin/integrations/nelc/stats`	MANAGER, ADMIN, SUPER_ADMIN	xAPI transmission stats
GET	`/admin/integrations/nelc/log`	MANAGER, ADMIN, SUPER_ADMIN	Paginated transmission log
GET	`/admin/integrations/nelc/log/:id`	MANAGER, ADMIN, SUPER_ADMIN	Statement detail + attempts
POST	`/admin/integrations/nelc/retry/:id`	MANAGER, ADMIN, SUPER_ADMIN	Re-enqueue a failed statement
POST	`/admin/integrations/nelc/test-connection`	MANAGER, ADMIN, SUPER_ADMIN	Send a probe statement
POST	`/admin/integrations/nelc/trigger/joined`	MANAGER, ADMIN, SUPER_ADMIN	Send NELC joined trigger
POST	`/admin/integrations/nelc/trigger/left`	MANAGER, ADMIN, SUPER_ADMIN	Send NELC left trigger

34. Zoom integration

Controller: `zoom/zoom.controller.ts` at `/api/zoom`.

Method	Path	Roles	Description
POST	`/zoom/webhook`	Public	Zoom webhook (HMAC signed)
POST	`/zoom/mock-webhook`	Public	Dev-only event simulator
GET	`/zoom/check-webinar-license`	MANAGER, ADMIN	Probe Zoom for user's webinar entitlement
GET	`/zoom/config`	ADMIN, SUPER_ADMIN	Read Zoom S2S config
PUT	`/zoom/config`	ADMIN, SUPER_ADMIN	Update Zoom S2S config
GET	`/zoom/account-info`	ADMIN, SUPER_ADMIN	Zoom account info
POST	`/zoom/config/test`	ADMIN, SUPER_ADMIN	Test Zoom connection

35. Storage

Controller: `storage/storage.controller.ts` at `/api/storage`.

Method	Path	Roles	Description
POST	`/storage/upload`	Public	Trainer document upload for public form (multipart, malware-scanned)
POST	`/storage/upload-avatar`	Authenticated	Avatar upload (JPEG / PNG / WebP)
GET	`/storage/files/:bucket/*`	Public	Local dev file serve with HTTP Range support

36. Admin settings and templates

Controllers: `admin-settings/admin-settings.controller.ts` (`/api/admin/settings`), `admin-settings/public-settings.controller.ts` (`/api/public`, covered in section 6), `templates/templates.controller.ts` (`/api/admin/templates`).

Method	Path	Roles	Description
GET	`/admin/settings`	ADMIN, SUPER_ADMIN	All settings snapshot
GET	`/admin/settings/:category`	ADMIN, SUPER_ADMIN	One category
PATCH	`/admin/settings/:category`	ADMIN, SUPER_ADMIN	Update one category
GET	`/admin/settings/templates/all`	ADMIN, SUPER_ADMIN	Notification templates list
GET	`/admin/settings/templates/:id`	ADMIN, SUPER_ADMIN	Template detail
POST	`/admin/settings/templates`	ADMIN, SUPER_ADMIN	Create template
PATCH	`/admin/settings/templates/:id`	ADMIN, SUPER_ADMIN	Update template
DELETE	`/admin/settings/templates/:id`	ADMIN, SUPER_ADMIN	Delete template
GET	`/admin/settings/integrations/health`	ADMIN, SUPER_ADMIN	Integrations last-success timestamps
GET	`/admin/templates`	ADMIN, SUPER_ADMIN	List overridable templates
GET	`/admin/templates/:eventKey/:locale`	ADMIN, SUPER_ADMIN	Read template body
PUT	`/admin/templates/:eventKey/:locale`	ADMIN, SUPER_ADMIN	Upsert template body
DELETE	`/admin/templates/:eventKey/:locale`	ADMIN, SUPER_ADMIN	Reset to default
POST	`/admin/templates/:eventKey/:locale/test-send`	ADMIN, SUPER_ADMIN	Send test render

37. Admin stats and monitoring

Controllers: `admin-stats/admin-stats.controller.ts` (`/api/admin/stats`), `monitoring/monitoring.controller.ts` (`/api/admin/monitoring`).

Method	Path	Roles	Description
GET	`/admin/stats/overview`	ADMIN, SUPER_ADMIN	Admin overview tiles
GET	`/admin/stats/dashboard`	ADMIN, SUPER_ADMIN	Dashboard data
GET	`/admin/stats/security/sessions`	ADMIN, SUPER_ADMIN	Active security sessions
GET	`/admin/stats/security/failed-logins`	ADMIN, SUPER_ADMIN	Failed-login timeline
GET	`/admin/stats/security/suspicious-ips`	ADMIN, SUPER_ADMIN	Suspicious IP list
GET	`/admin/stats/instructors-list`	ADMIN, SUPER_ADMIN	Instructor picker
GET	`/admin/stats/managers-list`	ADMIN, SUPER_ADMIN	Manager picker
GET	`/admin/monitoring/services-health`	ADMIN, SUPER_ADMIN	Services health matrix
GET	`/admin/monitoring/jobs-stats`	ADMIN, SUPER_ADMIN	BullMQ job stats
GET	`/admin/monitoring/cloudsql`	ADMIN, SUPER_ADMIN	Cloud SQL instance metrics
GET	`/admin/monitoring/redis`	ADMIN, SUPER_ADMIN	Redis instance metrics
GET	`/admin/monitoring/buckets`	ADMIN, SUPER_ADMIN	GCS bucket metrics
GET	`/admin/monitoring/uptime`	ADMIN, SUPER_ADMIN	Uptime probes

38. Backups and website recovery

Controllers: `backups/backups.controller.ts` (`/api/admin/backups`), `backups/website-recovery.controller.ts` (`/api/admin/backups/website`).

Method	Path	Roles	Description
GET	`/admin/backups/overview`	ADMIN, SUPER_ADMIN	Backups overview
GET	`/admin/backups`	ADMIN, SUPER_ADMIN	List backups
GET	`/admin/backups/restore-points`	ADMIN, SUPER_ADMIN	Restore points
GET	`/admin/backups/audit`	ADMIN, SUPER_ADMIN	Backup audit history
POST	`/admin/backups/snapshot`	ADMIN, SUPER_ADMIN	Create snapshot
POST	`/admin/backups/export`	ADMIN, SUPER_ADMIN	Create DB export
POST	`/admin/backups/files`	ADMIN, SUPER_ADMIN	Create files export
GET	`/admin/backups/files/deleted`	ADMIN, SUPER_ADMIN	Deleted-file recycle bin
POST	`/admin/backups/files/restore`	ADMIN, SUPER_ADMIN	Restore a deleted file
GET	`/admin/backups/materials/deleted`	ADMIN, SUPER_ADMIN	Deleted-material recycle bin
POST	`/admin/backups/materials/:id/restore`	ADMIN, SUPER_ADMIN	Restore a deleted material
GET	`/admin/backups/courses/deleted`	ADMIN, SUPER_ADMIN	Deleted-course recycle bin
POST	`/admin/backups/courses/:id/restore`	ADMIN, SUPER_ADMIN	Restore a deleted course
GET	`/admin/backups/restore-clones`	ADMIN, SUPER_ADMIN	Sandbox clones
POST	`/admin/backups/restore-clones`	ADMIN, SUPER_ADMIN	Create sandbox clone
POST	`/admin/backups/restore-clones/:id/extend`	ADMIN, SUPER_ADMIN	Extend sandbox clone expiry
POST	`/admin/backups/restore-clones/:id/drop`	ADMIN, SUPER_ADMIN	Drop sandbox clone
POST	`/admin/backups/:id/download`	ADMIN, SUPER_ADMIN	Get signed download URL
GET	`/admin/backups/website/config`	ADMIN, SUPER_ADMIN	Website recovery config

Method	Path	Roles	Description
GET	`/admin/backups/website/revisions`	ADMIN, SUPER_ADMIN	Website content revisions
POST	`/admin/backups/website/rollback`	ADMIN, SUPER_ADMIN	Rollback to a revision

Backups are restore-to-clone by design (no one-click in-place restore).

---

39. Service status and scheduled maintenance

Controllers: `service-status/service-status.controller.ts` (`/api/public/service-status` , `/api/service-status` , `/api/admin/service-status`), `scheduled-maintenance/scheduled-maintenance.controller.ts` (`/api/admin/maintenance/windows` , `/api/maintenance/banner`).

Method	Path	Roles	Description
GET	`/public/service-status`	Public	Live status snapshot
POST	`/public/service-status/view`	Public	Fire-and-forget view tracker
GET	`/public/service-status/summary`	Public	30-day transparency footer
GET	`/public/service-status/history`	Public	Paginated incident archive
GET	`/public/service-status/banner-incidents`	Public	Cross-dashboard banner feed
GET	`/public/service-status/active-for-link`	Public	Support "link incident" picker
POST	`/public/service-status/incidents/:id/link-ticket`	Public (authenticated)	Link ticket to incident
DELETE	`/public/service-status/incidents/:id/link-ticket/:ticketId`	Public (authenticated)	Unlink ticket
GET	`/public/service-status/tickets/:ticketId/incidents`	Public (authenticated)	Incidents linked to a ticket
GET	`/public/service-status/incidents/:id/tickets`	Public (authenticated)	Tickets linked to an incident
GET	`/service-status/components`	Authenticated	Component status list
PUT	`/service-status/components/:key`	ADMIN, SUPER_ADMIN	Set component status override
POST	`/service-status/components/:key/clear-override`	Authenticated	Clear component override
GET	`/admin/service-status/incidents`	ADMIN, SUPER_ADMIN	Admin incidents list
POST	`/admin/service-status/incidents`	ADMIN, SUPER_ADMIN	Create incident
POST	`/admin/service-status/incidents/:id/updates`	ADMIN, SUPER_ADMIN	Post incident update
DELETE	`/admin/service-status/incidents/:id`	ADMIN, SUPER_ADMIN	Delete incident
GET	`/admin/service-status/incidents/:id/report`	Authenticated	Post-mortem report
PATCH	`/admin/service-status/incidents/:id/report`	ADMIN, SUPER_ADMIN	Update post-mortem

Method	Path	Roles	Description
GET	`/admin/service-status/incidents/:id/tickets`	ADMIN, SUPER_ADMIN	Tickets linked to incident
GET	`/admin/service-status/analytics`	Authenticated	Analytics
GET	`/admin/service-status/incidents/export.csv`	Authenticated	CSV export
GET	`/admin/maintenance/windows`	ADMIN, SUPER_ADMIN	Maintenance windows list
GET	`/admin/maintenance/windows/next-upcoming`	ADMIN, SUPER_ADMIN	Next upcoming window
POST	`/admin/maintenance/windows`	ADMIN, SUPER_ADMIN	Create maintenance window
PATCH	`/admin/maintenance/windows/:id`	ADMIN, SUPER_ADMIN	Update window
DELETE	`/admin/maintenance/windows/:id`	ADMIN, SUPER_ADMIN	Cancel window
GET	`/maintenance/banner`	Authenticated	Current banner window

40. Error monitoring

Controllers: `error-monitoring/error-monitoring.controller.ts` (`/api/admin/error-monitoring`), `error-monitoring/error-monitoring-webhook.controller.ts` (`/api/webhooks/sentry`).

Method	Path	Roles	Description
GET	`/admin/error-monitoring/overview`	ADMIN, SUPER_ADMIN	Overview tiles
GET	`/admin/error-monitoring/top`	ADMIN, SUPER_ADMIN	Top issues
GET	`/admin/error-monitoring/trend`	ADMIN, SUPER_ADMIN	Trend series
GET	`/admin/error-monitoring/issue/:issueId`	ADMIN, SUPER_ADMIN	Issue detail
POST	`/admin/error-monitoring/issue/:issueId/resolve`	ADMIN, SUPER_ADMIN	Resolve issue
POST	`/webhooks/sentry/alert`	Public	Sentry alert receiver

41. Audit logs

Controller: `audit-logs/audit-logs.controller.ts` at `/api/audit-logs`.

Method	Path	Roles	Description
GET	`/audit-logs/finance/filters`	FINANCE, ADMIN, SUPER_ADMIN	Distinct entity types + actions
GET	`/audit-logs/finance/actors`	FINANCE, ADMIN, SUPER_ADMIN	Distinct actors
GET	`/audit-logs/finance/summary`	FINANCE, ADMIN, SUPER_ADMIN	KPI rollup for header
GET	`/audit-logs/finance/export`	FINANCE, ADMIN, SUPER_ADMIN	NELC-compliant export (CSV / XLSX / PDF)
GET	`/audit-logs/finance`	FINANCE, ADMIN, SUPER_ADMIN	Paginated finance log list
GET	`/audit-logs`	FINANCE, MANAGER, ADMIN	Per-entity lookup (entityType, entityId)
GET	`/audit-logs/admin/list`	ADMIN, SUPER_ADMIN	Broad admin search
GET	`/audit-logs/admin/distinct`	ADMIN, SUPER_ADMIN	Distinct filter values
GET	`/audit-logs/admin/export`	ADMIN, SUPER_ADMIN	CSV export

42. Health endpoints

Controller: `health/health.controller.ts` at `/api/health` (class-level `@Public()`).

Endpoint	Response
`GET /api/health`	200 while the process is alive
`GET /api/health/live`	200 always (Cloud Run liveness probe)
`GET /api/health/ready`	200 if DB + Redis reachable; 503 otherwise

Sample `/api/health/ready` response:

```
{
  "status": "ok",
  "timestamp": "2026-08-17T09:44:34.715Z",
  "checks": {
    "database": { "ok": true, "latencyMs": 7 },
    "redis": { "ok": true, "latencyMs": 17 }
  }
}
```


43. Webhook endpoints

Called by external systems. All webhook endpoints:

- Verify HMAC signature before enqueueing to BullMQ
- Return 200 fast to prevent gateway retry storms
- Deferred work happens in `physioguidance-api-worker`

Endpoint	Sender	Signature source
`POST /api/webhooks/:gateway` (`moyasar`, `tabby`, `tamara`)	Moyasar / Tabby / Tamara	`MOYASAR_WEBHOOK_SECRET`, `TABBY_WEBHOOK_SECRET`, `TAMARA_NOTIFICATION_TOKEN`
`POST /api/zoom/webhook`	Zoom	`ZOOM_WEBHOOK_SECRET_TOKEN`
`POST /api/webhooks/sentry/alert`	Sentry	Sentry alert rule secret

`POST /api/webhooks/:gateway` is a single dispatcher controller — it resolves the `:gateway` path parameter to the correct provider adapter.

44. Internal endpoints

Not for external consumption. Called by Cloud Tasks or Cloud Scheduler on the internal service URL.

Endpoint	Caller	Auth
`POST /api/internal/tasks/dispatch`	Cloud Tasks	`@Public()` + OIDC identity + dispatch token
`POST /api/payments/reconciliation/cron/daily-scan`	Cloud Scheduler	`@Public()` + `RECONCILIATION_CRON_TOKEN`
`POST /api/payments/reconciliation/cron/stale-reminders`	Cloud Scheduler	`@Public()` + cron token
`POST /api/payments/reconciliation/cron/abandoned-sweep`	Cloud Scheduler	`@Public()` + cron token
`POST /api/payments/reconciliation/cron/overdue-invoices`	Cloud Scheduler	`@Public()` + cron token
`POST /api/payments/reconciliation/cron/bnpl-status-poll`	Cloud Scheduler	`@Public()` + cron token

45. Component status matrix

Using Amna's four-category vocabulary.

Item	Status	Notes
REST API served at `/api/*`	Implemented	Global prefix set in `main.ts`
JWT auth + role-based authorization	Implemented	`JwtAuthGuard` + `RolesGuard` + `@Roles`
Rate limiting (global + per-route)	Implemented	NestJS Throttler, 600/min/IP + tighter on sensitive routes
Webhook signature verification	Implemented	HMAC verification for Moyasar, Tabby, Tamara, Zoom, Sentry
Health endpoints (`/health` , `/live` , `/ready`)	Implemented	Real dependency checks in `/ready`
Uniform error contract	Implemented	Nest built-in exception filter
Public + internal + protected route separation	Implemented	`@Public()` , `@Roles` , guard chain
Public verification endpoints (certificate, invoice)	Implemented	30/min throttle on certificate verify
Cloud Tasks dispatcher (`/internal/tasks/dispatch`)	Implemented	Cloud Tasks + OIDC + dispatch token
Cloud Scheduler cron endpoints	Implemented	Public with cron token + tighter per-route throttles
Trainer-application OTP flow (public intake)	Implemented	Captcha + email OTP + scoped JWT
API versioning	Not Applicable	No breaking-change versioning today; not requested in scope

46. Where to look next

- How auth cookies work end-to-end -> security-architecture.md section 5
- How webhooks flow through the worker -> system-architecture.md section 6
- How Cloud Tasks dispatches to internal endpoints -> gcp-deployment.md section 7
- Every env var and secret used -> environment-configuration-CONFIDENTIAL.md